

# Laboratorio 3 — Redes Neuronales Recurrentes y LSTM

CC3092 · Deep Learning y Sistemas Inteligentes — Esteban Cárcamo · Clasificación binaria de sentimiento sobre IMDB Reviews (50 000 reseñas, 2 clases balanceadas). Split: 20 000 train / 5 000 validación / 25 000 test (usado una sola vez).

## 1. Investigación: capas de PyTorch para RNN y LSTM

**nn.Embedding** — tabla de búsqueda entrenable que mapea un id de token a un vector denso; equivale a multiplicar un one-hot por una matriz  $|V| \times d$  pero implementado como indexación. Parámetros: `num_embeddings` (vocabulario), `embedding_dim`, y `padding_idx`, cuyo embedding se inicializa en cero y **nunca recibe gradiente** (lo usamos con `<pad>=0`). Es la capa con más parámetros:  $20\,000 \times 128 = 2.56\text{ M}$ , frente a  $\sim 132\text{ K}$  de una LSTM de 128 unidades.

**nn.RNN** — recurrencia de Elman,  $h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b)$ , con pesos compartidos en todos los pasos. Parámetros relevantes: `input_size`, `hidden_size`, `num_layers`, `nonlinearity` ('tanh'/'relu'), `batch_first`, `dropout` (solo *entre* capas apiladas) y `bidirectional`. Devuelve `output` (todos los pasos) y `h_n` (estado final por capa); en many-to-one usamos `h_n[-1]`.

**nn.LSTM** — misma interfaz, pero mantiene **dos** estados (oculto  $h_t$  y de celda  $c_t$ ) y tres compuertas sigmoideas (olvido  $f$ , entrada  $i$ , salida  $o$ ) con  $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$  y  $h_t = o_t \odot \tanh(c_t)$ . Tiene  $4 \times$  los parámetros de una RNN del mismo `hidden_size` y devuelve la tupla `(h_n, c_n)`.

**pad\_sequence / pack\_padded\_sequence / pad\_packed\_sequence** — la primera apila tensores de longitud distinta en un rectángulo `(B, Tmax)` rellenando con `padding_value`; la segunda lo convierte en un `PackedSequence` que hace que la recurrente **salte los <pad>** (`enforce_sorted=False` evita ordenar el batch a mano); la tercera revierte al rectángulo, necesaria solo si se usan las salidas de todos los pasos. **Sin packing, h\_T de una reseña de 40 tokens en un batch de 300 sería el resultado de digerir 260 pasos de relleno**; lo verificamos numéricamente en el notebook.

**clip\_grad\_norm** — calcula la norma global del gradiente y, si supera `max_norm`, reescala todos los gradientes por el mismo factor (preserva la dirección, acota la magnitud). Se llama **entre** `loss.backward()` y

`optimizer.step()`. Es la defensa estándar contra el exploding gradient.

**nn.Dropout en recurrentes** — el argumento `dropout=` de `nn.RNN/nn.LSTM` solo actúa *entre capas apiladas*: con `num_layers=1` no hace nada. Aplicar dropout en cada paso de tiempo con máscaras distintas inyecta ruido que se acumula y rompe la memoria. Por eso lo aplicamos donde sí es seguro: sobre los embeddings (antes de la recurrencia) y sobre  $h_T$  (antes de la lineal).

### Conceptos

**Estado oculto vs. de celda.**  $c_t$  es la memoria de largo plazo, actualizada de forma **aditiva**: la información puede viajar muchos pasos casi intacta si la compuerta de olvido lo permite.  $h_t$  es la vista filtrada de esa memoria que se expone al exterior ( $h_t = o_t \odot \tanh(c_t)$ ). Una RNN simple solo tiene  $h_t$ : su memoria y su salida son la misma cosa, no puede recordar algo sin usarlo ahora.

**Vanishing / exploding gradient.** En BPTT,  $\partial h_T / \partial h_k = \prod_{t=k+1..T} W_{hh}^T \cdot \text{diag}(\tanh')$ , esencialmente una potencia de matriz: si el factor por paso es  $<1$  el gradiente **decae exponencialmente** en  $T$ , si es  $>1$  explota. Es exponencial en la longitud: un factor 0.9 deja 0.5 % del gradiente a 50 pasos, pero  $10^{-14}$  a 300 — de ahí la severidad en secuencias largas.

**Cómo ayudan las compuertas.** La ruta  $c_{t-1} \rightarrow c_t$  es una suma con multiplicación elemento a elemento por  $f_t$ , no por una matriz de pesos: su derivada es simplemente  $f_t$ , y con  $f \approx 1$  el gradiente se propaga casi sin atenuación (un "camino de gradiente constante" análogo a una conexión residual). Además las compuertas son aprendidas y dependen de la entrada. El *exploding* no lo resuelven — para eso está el clipping —, pero el *vanishing* sí queda muy mitigado.

## 2. Datos y protocolo experimental

**Tokenización** word-level con regex (minúsculas, se elimina `<br />`, se conservan apóstrofes y los signos `! ?`). **Vocabulario**: 20 000 palabras más frecuentes de train (min\_freq 2) de 79 124 tipos; solo 2.75 % de los tokens de train y 3.49 % de los de test caen en `<unk>`. Construido **solo con train** para evitar leakage.

**Longitudes**: mín. 10, máx. 2 503, promedio 235, mediana 176 (cola derecha larga). **Padding** dinámico por batch: un batch es un tensor rectangular, así que hay que rellenar; con `max_len=300` se conserva el 81 % de los tokens y el 77 % de las reseñas completas a un tercio del costo del peor caso. Todos los modelos usan Adam, `BCEWithLogitsLoss`, batch 64, semilla 42, y **early stopping sobre val\_loss** restaurando el mejor epoch.

3. Iteraciones (15: 5 por arquitectura, cambiando una cosa a la vez)

| Iteración             | Qué cambia respecto a la anterior       | Params    | Epoch | val_loss | Acc    | Prec   | Rec    | F1     | t (s) |
|-----------------------|-----------------------------------------|-----------|-------|----------|--------|--------|--------|--------|-------|
| mlp-01-base           | emb 64, cabeza (64), sin regularización | 1 284 225 | 4/7   | 0.3216   | 0.8724 | 0.8726 | 0.8724 | 0.8723 | 5.0   |
| mlp-02-capacidad      | emb 128, cabeza (256,128)               | 2 626 049 | 3/6   | 0.3172   | 0.8768 | 0.8768 | 0.8768 | 0.8768 | 5.2   |
| mlp-03-dropout        | + dropout 0.5                           | 2 626 049 | 4/7   | 0.3172   | 0.8748 | 0.8749 | 0.8748 | 0.8748 | 6.0   |
| mlp-04-weight-decay   | + weight decay 1e-4                     | 2 626 049 | 7/10  | 0.3061   | 0.8764 | 0.8764 | 0.8764 | 0.8764 | 9.7   |
| mlp-05-lr-alto        | lr 1e-3 → 1e-2 (sin weight decay)       | 2 626 049 | 1/4   | 0.3002   | 0.8814 | 0.8814 | 0.8814 | 0.8814 | 3.7   |
| rnn-01-base           | 1 capa, sin dropout ni clipping         | 2 593 153 | 6/10  | 0.5849   | 0.7140 | 0.7205 | 0.7140 | 0.7113 | 34.9  |
| rnn-02-clip           | + gradient clipping 1.0                 | 2 593 153 | 4/8   | 0.5302   | 0.7514 | 0.7515 | 0.7514 | 0.7513 | 28.0  |
| rnn-03-lr-bajo        | lr 1e-3 → 3e-4                          | 2 593 153 | 12/16 | 0.4556   | 0.8160 | 0.8160 | 0.8160 | 0.8159 | 55.1  |
| rnn-04-hidden-256     | hidden 128 → 256, dropout 0.3           | 2 659 073 | 16/20 | 0.4678   | 0.7980 | 0.7980 | 0.7980 | 0.7980 | 72.5  |
| rnn-05-bidireccional  | bidireccional, dropout 0.3              | 2 626 305 | 8/12  | 0.5133   | 0.7560 | 0.7562 | 0.7560 | 0.7558 | 63.9  |
| lstm-01-base          | espejo exacto de rnn-01                 | 2 692 225 | 7/11  | 0.3869   | 0.8562 | 0.8563 | 0.8562 | 0.8562 | 39.6  |
| lstm-02-clip          | + gradient clipping 1.0                 | 2 692 225 | 5/9   | 0.3687   | 0.8532 | 0.8532 | 0.8532 | 0.8532 | 33.9  |
| lstm-03-dropout       | + dropout 0.5                           | 2 692 225 | 11/15 | 0.4119   | 0.8694 | 0.8700 | 0.8694 | 0.8692 | 55.6  |
| lstm-04-hidden-256    | hidden 128 → 256                        | 2 955 521 | 8/12  | 0.3354   | 0.8710 | 0.8714 | 0.8710 | 0.8708 | 54.9  |
| lstm-05-bidireccional | bidireccional + weight decay 1e-5       | 3 351 041 | 7/11  | 0.3438   | 0.8704 | 0.8709 | 0.8704 | 0.8700 | 81.2  |

Métricas sobre **validación** en el epoch seleccionado por early stopping; precision/recall/F1 macro. En verde, la mejor configuración de cada arquitectura (criterio: val\_loss). Las curvas de pérdida train/val de las 15 iteraciones están en el notebook.

4. Comparación de arquitecturas (test, una única evaluación)

| Arquitectura   | Config. ganadora   | Params    | % en embedding | t entren. | val acc | test acc | test prec | test rec | test F1 | FP / FN     |
|----------------|--------------------|-----------|----------------|-----------|---------|----------|-----------|----------|---------|-------------|
| MLP (baseline) | mlp-05-lr-alto     | 2 626 049 | 97.5 %         | 3.7 s     | 0.8814  | 0.8684   | 0.8684    | 0.8684   | 0.8684  | 1637 / 1654 |
| RNN simple     | rnn-03-lr-bajo     | 2 593 153 | 98.7 %         | 55.1 s    | 0.8160  | 0.8040   | 0.8044    | 0.8040   | 0.8039  | 2680 / 2221 |
| LSTM           | lstm-04-hidden-256 | 2 955 521 | 86.6 %         | 54.9 s    | 0.8710  | 0.8656   | 0.8669    | 0.8656   | 0.8655  | 2051 / 1309 |

| Sección 4.1 — max_len | 50     | 200    | 500    | Ganancia 50 → 500               |
|-----------------------|--------|--------|--------|---------------------------------|
| RNN — val accuracy    | 0.7214 | 0.7908 | 0.8228 | +10.1 puntos                    |
| LSTM — val accuracy   | 0.7720 | 0.8520 | 0.8884 | +11.6 puntos                    |
| Brecha LSTM – RNN     | +5.1   | +6.1   | +6.6   | la brecha crece con la longitud |

**Parámetros vs. calidad: no hay relación.** Los tres modelos rondan 2.6–3.0 M de parámetros y el 87–99 % están en la capa de embedding, que es idéntica en los tres. Los 6 puntos de diferencia los deciden decenas de miles de parámetros de recurrencia. La RNN, el modelo más pequeño, es la peor; la LSTM, el más grande, no gana. **Costo:** el MLP entrena 13–15× más rápido porque su cómputo es una suma paralelizable sobre el tiempo, mientras que las recurrentes ejecutan 300 pasos inherentemente secuenciales — la GPU paraleliza el batch, no el tiempo.

## 5. Análisis de resultados

**Hiperparámetro de mayor impacto.** *MLP*: positivo, subir el lr a  $1e-2$  (+0.7 pts, mejor val\_loss del grupo) — resultado inesperado que se explica leyendo la curva: el modelo alcanza su óptimo de generalización en el **primer epoch** y el early stopping se queda ahí, así que la configuración ganadora es también la más frágil. Negativo: dropout 0.5 (−0.2 pts). *RNN*: positivo, bajar el lr a  $3e-4$  (**+6.5 pts**) y activar clipping (+3.7 pts) — juntos, casi 10 puntos, el efecto más grande del laboratorio; negativo, subir hidden a 256 (−1.8 pts) y la versión bidireccional (−6 pts). *LSTM*: positivo, hidden  $128 \rightarrow 256$  (−0.077 de val\_loss, el mayor salto en la métrica de selección); negativo, dropout 0.5 aislado (+0.043 de val\_loss). **El mismo cambio de capacidad que perjudica a la RNN beneficia a la LSTM**: la RNN no está limitada por capacidad sino por su capacidad de entrenarse.

**¿Ayudó la regularización?** Solo cuando ataca el problema real. El **MLP** es el único que sobreajusta de verdad y ahí el ganador fue el **weight decay** (val\_loss  $0.3172 \rightarrow 0.3061$ ), no el dropout: el sobreajuste vive en la matriz de embeddings y el weight decay penaliza exactamente esos pesos. En la *RNN* el método que rindió no regulariza nada: el **gradient clipping** (+3.7 pts) combate inestabilidad numérica, no sobreajuste — la *RNN* *underfitte*. En la *LSTM*, clipping ligeramente positivo y dropout rentable solo *combinado* con más capacidad.

**MLP vs. RNN vs. LSTM en test** ( $0.8684 > 0.8656 \gg 0.8040$ ). La lectura ingenua —"gana el modelo que ignora el orden, luego el orden no importa"— es incorrecta. La señal de sentimiento en IMDB es en gran medida **léxica**, y promediar embeddings es un estimador robusto y baratísimo de esa señal: el MLP alcanza el techo léxico de forma trivial. La LSTM llega al mismo sitio pero teniendo que *aprender* a hacerlo a través de una recurrencia, y con 20 000 ejemplos y embeddings desde cero no tiene datos para explotar del todo su ventaja — aunque con max\_len=500 llega a **0.8884** en validación, por encima de cualquier MLP de la rejilla. La comparación limpia es **RNN vs. LSTM**, idénticas salvo por las compuertas: **+6.2 puntos** en test. Ese es, aislado, el valor de la memoria protegida.

**¿La RNN pierde desempeño con secuencias largas?** No en términos absolutos, pero sí relativos. Ambas mejoran al alargar la secuencia porque a 50 tokens el factor dominante es la **pérdida de información** (solo se ve el 21 % de los tokens del corpus), y ese efecto enmascara cualquier degradación. La señal de vanishing gradient está en la **brecha creciente**:  $+5.1 \rightarrow +6.1 \rightarrow +6.6$  puntos a favor de la LSTM. La RNN convierte cada token adicional en menos desempeño. Es exactamente lo

que predice la teoría: a 500 pasos, el producto de jacobianos deja sin señal de aprendizaje al principio de la reseña, así que la RNN entrena de facto solo su final, mientras que la ruta aditiva de la celda LSTM mantiene el gradiente vivo mucho más lejos. Dos evidencias colaterales apuntan a lo mismo: el clipping le dio +3.7 pts a la RNN y casi nada a la LSTM, y la capacidad extra ayudó solo a la LSTM.

**¿Dónde se equivoca cada modelo?** (i) **Todos caen con las reseñas largas**: de  $<100$  a  $>500$  tokens, la accuracy baja de  $0.878 \rightarrow 0.801$  en el MLP,  $0.885 \rightarrow 0.799$  en la LSTM y  $0.827 \rightarrow 0.709$  en la RNN. Parte de la caída es truncamiento (con max\_len 300, las reseñas de  $>500$  se ven a medias), pero **la RNN cae 1.5× más rápido** que las otras dos. (ii) **La LSTM tiene un sesgo asimétrico**: 2 051 falsos positivos contra 1 309 falsos negativos — le cuesta reconocer las negativas, coherente con que las reseñas negativas de IMDB describen la trama en tono neutro y guardan el veredicto para el final, mientras las positivas reparten adjetivos entusiastas por todo el texto. El MLP, que promedia, falla de forma casi simétrica (1 637 / 1 654). (iii) **1 374 reseñas (5.5 % del test) fallan en los tres modelos** y comparten un patrón claro: **sentimiento mixto con veredicto tardío** — párrafos elogiando la actuación y la fotografía, y al final la conclusión de que la película no funciona; la etiqueta viene de la calificación en estrellas, así que es negativa aunque el 80 % del texto sea positivo. Ninguna arquitectura lo resuelve: el MLP porque promedia (gana el volumen de palabras positivas) y las recurrentes porque ese veredicto suele caer **después del corte de 300 tokens**. El sarcasmo produce el mismo tipo de error.

**¿Qué desplegaríamos en producción?** Priorizando exactitud y eficiencia y para este problema concreto, **el MLP sobre embeddings promediados**: empata a la LSTM en test con 13× menos tiempo de entrenamiento, inferencia trivialmente paralelizable (una suma, no 300 pasos secuenciales), sin estado que arrastrar y servible en CPU. Cambiaríamos de opinión en dos casos: si el dominio dependiera del orden (negación, sarcasmo, veredicto tardío, más clases), donde una LSTM con la secuencia completa es el mínimo viable; y si hubiera presupuesto para un **Transformer preentrenado**, que cambia la respuesta por completo — un DistilBERT afinado llega a ~93 % en IMDB porque trae conocimiento lingüístico de miles de millones de tokens en vez de aprender el idioma desde cero con 20 000 reseñas, y además su atención **se paraleliza sobre el tiempo**, resolviendo a la vez el problema de calidad y el cuello de botella secuencial que aquí nos obligó a elegir. Su costo es de inferencia (dos órdenes de magnitud más de cómputo por reseña) y de memoria.

## 6. Conclusiones

- **Las compuertas valen 6.2 puntos.** RNN y LSTM configuradas de forma idéntica salvo por el tipo de celda: 0.8040 vs 0.8656 en test. Es la medición más limpia del laboratorio.
- **La RNN simple está limitada por la optimización, no por la capacidad.** Sus dos mayores mejoras fueron bajar el lr (+6.5 pts) y el clipping (+3.7 pts); darle más capacidad la empeoró. La LSTM se comporta al revés en ambos casos.
- **El vanishing gradient se observa como brecha creciente, no como caída absoluta:** de 50 a 500 tokens ambas mejoran, pero la ventaja de la LSTM crece de +5.1 a +6.6 puntos.
- **Un baseline sin noción de orden es competitivo en sentiment analysis:** 0.8684 en test entrenando en 4 segundos. Vale la pena entrenarlo siempre antes de asumir que hace falta una arquitectura secuencial.
- **El conteo de parámetros no explica nada aquí:** 87–99 % de los pesos están en el embedding compartido; los 6 puntos de diferencia los deciden decenas de miles de parámetros de recurrencia.
- **El techo real son las reseñas de sentimiento mixto** (5.5 % del test falla en los tres modelos). Superarlas exige más contexto y modelar la estructura retórica del texto — terreno de los Transformers preentrenados.

**Repositorio:** <https://github.com/ecarcamo/CC3092-DEEP-LEARNING/tree/main/lab3> — notebook completo y comentado en [lab3/notebooks/lab3\\_rnn\\_lstm.ipynb](#).